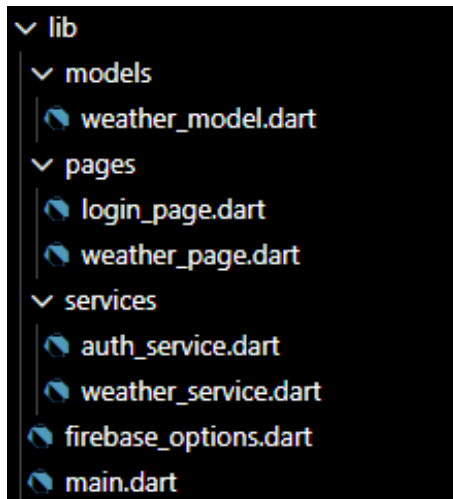




NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

## Ujicoba API

Project ini menggabungkan dua aplikasi Flutter menjadi satu, yaitu aplikasi Login dengan Firebase Authentication dan aplikasi Weather API accuweather. Terdapat 6 file Dart utama yang masing-masing memiliki tanggung jawab berbeda sesuai dengan prinsip pemisahan tanggung jawab (separation of concerns).



### 1. main.dart

File main.dart adalah titik masuk dari seluruh aplikasi Flutter. File ini bertanggung jawab untuk menginisialisasi Firebase, mendefinisikan tema aplikasi, dan menentukan halaman awal yang ditampilkan berdasarkan status login pengguna.

#### 1.1. Fungsi main()

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(const MyApp());  
}
```

WidgetsFlutterBinding.ensureInitialized() dipanggil pertama kali untuk memastikan Flutter sudah siap sebelum menjalankan kode asinkron. Firebase.initializeApp() menginisialisasi koneksi ke Firebase menggunakan konfigurasi dari firebase\_options.dart. Fungsi ini bersifat async sehingga harus menggunakan await agar proses inisialisasi selesai sebelum aplikasi berjalan.

#### 1.2. Class MyApp



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Weather App',  
      debugShowCheckedModeBanner: false,  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(  
          seedColor: const Color(0xFF1E3A5F),  
          brightness: Brightness.dark,  
        ), // ColorScheme.fromSeed  
        useMaterial3: true,  
      ), // ThemeData  
      home: const AuthWrapper(),  
    ); // MaterialApp  
  }  
}
```

MyApp merupakan widget utama yang bersifat StatelessWidget karena tidak memiliki state yang berubah. MaterialApp mendefinisikan tema gelap (dark mode) dengan warna utama biru tua (0xFF1E3A5F) dan menggunakan Material Design 3. Home diset ke AuthWrapper yang bertugas memeriksa status login.

### 1.3. Class AuthWrapper



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
// Mengecek status login user
// Jika sudah login -> WeatherPage, jika belum -> LoginPage
class AuthWrapper extends StatelessWidget {
  const AuthWrapper({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        // Menunggu koneksi Firebase
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(
            backgroundColor: Color(0xFF1E3A5F),
            body: Center(
              child: CircularProgressIndicator(color: Colors.white),
            ), // Center
          ); // Scaffold
        }

        // Jika user sudah login
        if (snapshot.hasData && snapshot.data != null) {
          return const WeatherPage();
        }

        // Jika user belum login
        return const LoginPage();
      },
    ); // StreamBuilder
  }
}
```

AuthWrapper menggunakan StreamBuilder untuk mendengarkan perubahan status autentikasi secara real-time dari Firebase. `authStateChanges()` mengembalikan Stream yang memancarkan nilai setiap kali status login berubah. Jika `snapshot.hasData` bernilai true berarti pengguna sudah login dan aplikasi akan menampilkan WeatherPage. Sebaliknya, jika belum login maka akan ditampilkan LoginPage. Selama menunggu respons Firebase, ditampilkan indikator loading.

## 2. `firebase_options.dart`

File ini berisi konfigurasi koneksi antara aplikasi Flutter dan project Firebase. Setiap platform (Android, iOS, Web) memiliki konfigurasi tersendiri karena masing-masing menggunakan APP ID yang berbeda.

### 2.1. Struktur Konfigurasi



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
class DefaultFirebaseOptions {  
  static FirebaseOptions get currentPlatform {  
    if (kIsWeb) return web;  
    switch (defaultTargetPlatform) {  
      case TargetPlatform.android:  
        return android;  
      case TargetPlatform.iOS:  
        return ios;  
      default:  
        throw UnsupportedError(  
          'DefaultFirebaseOptions belum dikonfigurasi untuk platform ini.',  
        );  
    }  
  }  
}
```

Property `currentPlatform` bersifat getter yang secara otomatis mendeteksi platform yang sedang digunakan dan mengembalikan konfigurasi yang sesuai. `kIsWeb` memeriksa apakah aplikasi berjalan di browser web. `defaultTargetPlatform` mendeteksi platform mobile (Android/iOS).

## 2.2. Parameter Konfigurasi Firebase

```
static const FirebaseOptions android = FirebaseOptions(  
  apiKey: 'AIzaSyB7zdOXor4wFU6sSdiddi09Th-4Bo5MGfE',  
  appId: '1:947122131267:android:46e5d0370b916f24701ff6',  
  messagingSenderId: '947122131267',  
  projectId: 'weather-app-ec9d9',  
  storageBucket: 'weather-app-ec9d9.firebaseio.com',  
);
```

`apiKey` digunakan untuk mengidentifikasi project Firebase saat melakukan request API. `appId` adalah identifier unik aplikasi yang terdaftar di Firebase Console. `messagingSenderId` digunakan untuk layanan push notification. `projectId` adalah nama unik project Firebase. `storageBucket` adalah URL bucket penyimpanan Firebase Storage.

## 3. auth\_service.dart

File ini berisi semua logika yang berhubungan dengan autentikasi pengguna menggunakan Firebase Authentication. Pola desain yang digunakan adalah Service Layer, di mana semua operasi terkait autentikasi dikelompokkan dalam satu class.

### 3.1. Inisialisasi

```
class AuthService {  
  final FirebaseAuth _auth = FirebaseAuth.instance;  
  
  // Mendapatkan user yang sedang login  
  User? get currentUser => _auth.currentUser;  
  
  // Stream perubahan status auth  
  Stream<User?> get authStateChanges => _auth.authStateChanges();  
}
```

`FirebaseAuth.instance` mengambil instance singleton dari Firebase Authentication. `currentUser` adalah getter yang mengembalikan pengguna yang sedang login, atau null jika belum login. `authStateChanges` adalah Stream yang terus memantau perubahan status login secara real-time.



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

### 3.2. Fungsi Login

```
// Login dengan email dan password
Future<UserCredential?> signInWithEmail(String email, String password) async {
  try {
    return await _auth.signInWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );
  } on FirebaseAuthException catch (e) {
    throw _handleAuthException(e);
  }
}
```

Fungsi `signInWithEmail` menerima email dan password sebagai parameter. `email.trim()` dan `password.trim()` digunakan untuk menghapus spasi di awal/akhir string yang tidak disengaja. Jika login berhasil, fungsi mengembalikan `UserCredential`. Jika gagal, exception `FirebaseAuthException` ditangkap dan dikonversi menjadi pesan error yang mudah dipahami oleh pengguna.

### 3.3. Fungsi Register

```
// Daftar akun baru dengan email dan password
Future<UserCredential?> registerWithEmail(
  String email, String password) async {
  try {
    return await _auth.createUserWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );
  } on FirebaseAuthException catch (e) {
    throw _handleAuthException(e);
  }
}
```

Fungsi `registerWithEmail` membuat akun baru di Firebase Authentication menggunakan `createUserWithEmailAndPassword`. Proses ini sama dengan login namun menggunakan method yang berbeda. Firebase akan otomatis menyimpan data pengguna dan mengembalikan `UserCredential` jika berhasil.

### 3.4. Fungsi Logout & Error Handler



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
// Logout
Future<void> signOut() async {
  | await _auth.signOut();
  | }

// Konversi kode error Firebase ke pesan yang mudah dipahami
String _handleAuthException(FirebaseAuthException e) {
  | switch (e.code) {
  |   case 'user-not-found':
  |     | return 'Akun tidak ditemukan. Silakan daftar terlebih dahulu.';
  |   case 'wrong-password':
  |     | return 'Password salah. Coba lagi.';
  |   case 'email-already-in-use':
  |     | return 'Email sudah digunakan. Silakan login.';
  |   case 'weak-password':
  |     | return 'Password terlalu lemah. Minimal 6 karakter.';
  |   case 'invalid-email':
  |     | return 'Format email tidak valid.';
  |   case 'user-disabled':
  |     | return 'Akun ini telah dinonaktifkan.';
  |   case 'too-many-requests':
  |     | return 'Terlalu banyak percobaan. Coba lagi nanti.';
  |   default:
  |     | return 'Terjadi kesalahan: ${e.message}';
  | }
  | }
}
```

signOut() memanggil \_auth.signOut() yang akan membersihkan sesi pengguna dari perangkat dan Firebase secara bersamaan. \_handleAuthException() menggunakan switch-case untuk mengkonversi kode error Firebase (seperti 'user-not-found') menjadi pesan yang lebih mudah dipahami pengguna dalam Bahasa Indonesia.

#### 4. weather\_service.dart

File ini bertugas melakukan pemanggilan ke OpenWeatherMap API untuk mengambil data cuaca berdasarkan nama kota. Menggunakan package http untuk melakukan HTTP GET request.

##### 4.1. Konfigurasi API

```
class WeatherService {
  | static const String _apiKey = 'apiKey';
  | static const String _baseUrl =
  |   | 'https://api.openweathermap.org/data/2.5/weather';
}
```

\_apiKey adalah kunci API dari OpenWeatherMap yang digunakan untuk autentikasi setiap request. Dideklarasikan sebagai static const karena nilainya tidak berubah dan dapat diakses tanpa membuat instance class. \_baseUrl adalah URL dasar endpoint API cuaca.

##### 4.2. Fungsi getWeatherByCity



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
// Ambil cuaca berdasarkan nama kota
Future<WeatherModel> getWeatherByCity(String city) async {
  final url = Uri.parse(
    '$_baseUrl?q=$city&appid=$_apiKey&units=metric&lang=id',
  );

  final response = await http.get(url);

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    return WeatherModel.fromJson(data);
  } else if (response.statusCode == 404) {
    throw Exception('Kota tidak ditemukan. Periksa kembali nama kota.');
```

Uri.parse() membuat objek URI dari string URL dengan query parameters: q adalah nama kota, appid adalah API key, units=metric mengatur satuan suhu ke Celsius, dan lang=id mengatur bahasa deskripsi cuaca ke Bahasa Indonesia. http.get(url) melakukan HTTP GET request secara asinkron. statusCode 200 berarti sukses, 404 berarti kota tidak ditemukan. jsonDecode() mengurai respons JSON menjadi Map Dart yang kemudian diteruskan ke WeatherModel.fromJson().

## 5. weather\_model.dart

File ini mendefinisikan struktur data (model) untuk menyimpan informasi cuaca yang diterima dari API. Menggunakan pola Model/Entity yang umum dalam arsitektur aplikasi Flutter.

### 5.1. Deklarasi Properties

```
class WeatherModel {
  final String cityName;
  final String country;
  final double temperature;
  final double feelsLike;
  final double tempMin;
  final double tempMax;
  final int humidity;
  final double windSpeed;
  final String description;
  final String icon;
  final int visibility;
  final int pressure;
```

Semua properties dideklarasikan sebagai final karena data cuaca bersifat tidak berubah setelah dibuat. Tipe data disesuaikan dengan format data dari API: String untuk teks, double untuk angka desimal, int untuk angka bulat.

### 5.2. Factory Constructor fromJson



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
factory WeatherModel.fromJson(Map<String, dynamic> json) {  
  return WeatherModel(  
    cityName: json['name'] ?? '',  
    country: json['sys']['country'] ?? '',  
    temperature: (json['main']['temp'] as num).toDouble(),  
    feelsLike: (json['main']['feels_like'] as num).toDouble(),  
    tempMin: (json['main']['temp_min'] as num).toDouble(),  
    tempMax: (json['main']['temp_max'] as num).toDouble(),  
    humidity: json['main']['humidity'] ?? 0,  
    windSpeed: (json['wind']['speed'] as num).toDouble(),  
    description: json['weather'][0]['description'] ?? '',  
    icon: json['weather'][0]['icon'] ?? '01d',  
    visibility: json['visibility'] ?? 0,  
    pressure: json['main']['pressure'] ?? 0,  
  );  
}
```

Factory constructor fromJson bertugas mengkonversi data JSON dari API menjadi objek WeatherModel. Operator ?? memberikan nilai default jika data dari API bernilai null. (json['main']['temp'] as num).toDouble() mengkonversi tipe data num dari JSON ke double Dart karena JSON tidak membedakan int dan double. json['weather'][0]['description'] mengakses elemen pertama dari array weather dalam respons JSON.

### 5.3. Getter iconUrl

```
// URL icon cuaca dari OpenWeatherMap  
String get iconUrl => 'https://openweathermap.org/img/wn/$icon@2x.png';  
}
```

iconUrl adalah computed property (getter) yang menghasilkan URL lengkap untuk menampilkan ikon cuaca dari server OpenWeatherMap. @2x menandakan resolusi 2x (Retina) untuk tampilan yang lebih tajam di layar beresolusi tinggi.

## 6. login\_page.dart

File ini berisi tampilan dan logika untuk halaman login dan register. Menggunakan StatefulWidget karena terdapat state yang berubah seperti status loading, toggle login/register, dan visibilitas password.

### 6.1. State Variables

```
class _LoginPageState extends State<LoginPage> {  
  final AuthService _authService = AuthService();  
  
  final TextEditingController _emailController = TextEditingController();  
  final TextEditingController _passwordController = TextEditingController();  
  
  bool _isLogin = true; // toggle antara Login dan Register  
  bool _isLoading = false; // menampilkan loading indicator  
  bool _obscurePassword = true; // menyembunyikan/menampilkan password  
}
```

TextEditingController digunakan untuk mengambil dan mengontrol teks yang diketik pengguna pada field email dan password. \_isLogin menentukan apakah form



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

sedang dalam mode login atau register. `_isLoading` mencegah pengguna mengklik tombol berkali-kali saat proses sedang berjalan.

#### 6.2. Fungsi `_handleSubmit`

```
// Proses login atau register
Future<void> _handleSubmit() async {
  final email = _emailController.text.trim();
  final password = _passwordController.text.trim();

  if (email.isEmpty || password.isEmpty) {
    _showSnackBar('Email dan password tidak boleh kosong.');
```

*Marsya Zulfa*

```
    return;
  }

  setState(() => _isLoading = true);

  try {
    if (_isLogin) {
      await _authService.signInWithEmail(email, password);
    } else {
      await _authService.registerWithEmail(email, password);
    }
    // Navigasi ditangani otomatis oleh AuthWrapper di main.dart
  } catch (e) {
    _showSnackBar(e.toString());
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

Fungsi ini menangani proses submit form. Validasi dasar dilakukan terlebih dahulu sebelum memanggil API. `setState(() => _isLoading = true)` mengaktifkan loading indicator dan menonaktifkan tombol. `if (mounted)` pada blok `finally` memastikan widget masih ada di pohon widget sebelum memanggil `setState`, untuk menghindari error jika halaman sudah berpindah. Navigasi ke `WeatherPage` ditangani otomatis oleh `AuthWrapper` melalui stream `Firestore`.

#### 6.3. Tampilan UI



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [
            Color(0xFF1E3A5F),
            Color(0xFF0D1B2A),
            Color(0xFF1B4F72),
          ],
        ), // LinearGradient
      ), // BoxDecoration
      child: SafeArea(
        child: Center(
          child: SingleChildScrollView( // SingleChildScrollView ...
          ), // Center
        ), // SafeArea
      ), // Container
    ); // Scaffold
  }
}
```

Latar belakang menggunakan LinearGradient dengan tiga warna biru tua untuk menciptakan efek gradasi yang elegan. SafeArea memastikan konten tidak tertutup oleh notch atau status bar perangkat. SingleChildScrollView memungkinkan halaman di-scroll ketika keyboard muncul sehingga form tetap terlihat.

## 7. weather\_page.dart

File ini adalah halaman utama yang ditampilkan setelah pengguna berhasil login. Menampilkan informasi cuaca lengkap dan menyediakan fitur pencarian kota serta tombol logout.

### 7.1. State Variables & initState

```
class _WeatherPageState extends State<WeatherPage> {
  final AuthService _authService = AuthService();
  final WeatherService _weatherService = WeatherService();
  final TextEditingController _cityController = TextEditingController();

  WeatherModel? _weather; // data cuaca (null = belum dimuat)
  bool _isLoading = false; // status loading
  String? _errorMessage; // pesan error (null = tidak ada error)
  String _currentCity = 'Malang'; // Kota default saat pertama buka

  @override
  void initState() {
    super.initState();
    _fetchWeather(_currentCity); // langsung ambil cuaca saat halaman dibuka
  }
}
```

initState() dipanggil sekali saat widget pertama kali dibuat. Di sini digunakan untuk otomatis mengambil data cuaca kota Malang saat halaman pertama dibuka. WeatherModel? menggunakan tanda tanya yang berarti nullable — nilai awalnya null sebelum data API diterima.



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

### 7.2. Fungsi \_fetchWeather

```
Future<void> _fetchWeather(String city) async {  
  setState(() {  
    _isLoading = true;  
    _errorMessage = null;  
  });  
  
  try {  
    final weather = await _weatherService.getWeatherByCity(city);  
    setState(() {  
      _weather = weather;  
      _currentCity = city;  
    });  
  } catch (e) {  
    setState(() => _errorMessage = e.toString().replaceAll('Exception: ', ''));  
  } finally {  
    if (mounted) setState(() => _isLoading = false);  
  }  
}
```

Fungsi ini memanggil WeatherService untuk mengambil data cuaca. \_errorMessage di-reset ke null setiap kali request baru dimulai agar error sebelumnya tidak ditampilkan. replaceAll('Exception: ', '') membersihkan prefix 'Exception: ' dari pesan error Dart agar tampil lebih rapi kepada pengguna.

### 7.3. Fungsi Logout

```
// Tampilkan dialog konfirmasi logout  
void _showLogoutDialog() {  
  showDialog(  
    context: context,  
    builder: (ctx) => AlertDialog(  
      backgroundColor: const Color(0xFF1B4F72),  
      title: const Text('Logout', style: TextStyle(color: Colors.white)),  
      content: const Text(  
        'Apakah kamu yakin ingin keluar?',  
        style: TextStyle(color: Colors.white70),  
      ), // Text  
      actions: [  
        TextButton(  
          onPressed: () => Navigator.pop(ctx),  
          child: const Text('Batal', style: TextStyle(color: Colors.white60)),  
        ), // TextButton  
        TextButton(  
          onPressed: () {  
            Navigator.pop(ctx);  
            _handleLogout();  
          },  
          child: const Text(  
            'Logout',  
            style: TextStyle(color: Colors.redAccent),  
          ), // Text  
        ), // TextButton  
      ],  
    ), // AlertDialog  
  );  
}
```



NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

showDialog menampilkan dialog konfirmasi sebelum logout untuk mencegah logout yang tidak disengaja. Setelah \_authService.signOut() dipanggil, Firebase akan memperbarui authStateChanges stream yang didengarkan oleh AuthWrapper di main.dart, sehingga aplikasi otomatis berpindah ke LoginPage tanpa perlu navigasi manual.

#### 7.4. Tampilan Konten Cuaca

```
Widget _buildWeatherContent() {  
  final w = _weather!;  
  
  return SingleChildScrollView(  
    padding: const EdgeInsets.symmetric(horizontal: 20),  
    child: Column(  
      children: [  
        // Kota & Negara  
        Text(  
          '${w.cityName}, ${w.country}',  
          style: const TextStyle(  
            color: Colors.white,  
            fontSize: 28,  
            fontWeight: FontWeight.bold,  
          ), // TextStyle  
          textAlign: TextAlign.center,  
        ), // Text  
        const SizedBox(height: 4),  
        Text(  
          w.description.toUpperCase(),  
          style: TextStyle(  
            color: Colors.white.withOpacity(0.7),  
            fontSize: 14,  
            letterSpacing: 2,  
          ), // TextStyle  
        ), // Text  
      ],  
    ),  
  );  
}
```

\_buildWeatherContent() dipisah menjadi fungsi tersendiri untuk menjaga fungsi build() tetap bersih dan mudah dibaca. w.description.toUpperCase() mengubah deskripsi cuaca menjadi huruf kapital semua untuk efek visual. Image.network() menampilkan ikon cuaca langsung dari server OpenWeatherMap. GridView.count() dengan crossAxisCount: 2 menampilkan empat kartu informasi tambahan dalam grid 2 kolom.

#### 7.5. Logika Tampilan Bersyarat

```
// Konten Cuaca  
Expanded(  
  child: _isLoading  
    ? const Center(  
      child: CircularProgressIndicator(color: Colors.white),  
    ) // Center  
    : _errorMessage != null  
      ? _buildErrorWidget()  
      : _weather != null  
        ? _buildWeatherContent()  
        : const SizedBox(),  
), // Expanded
```

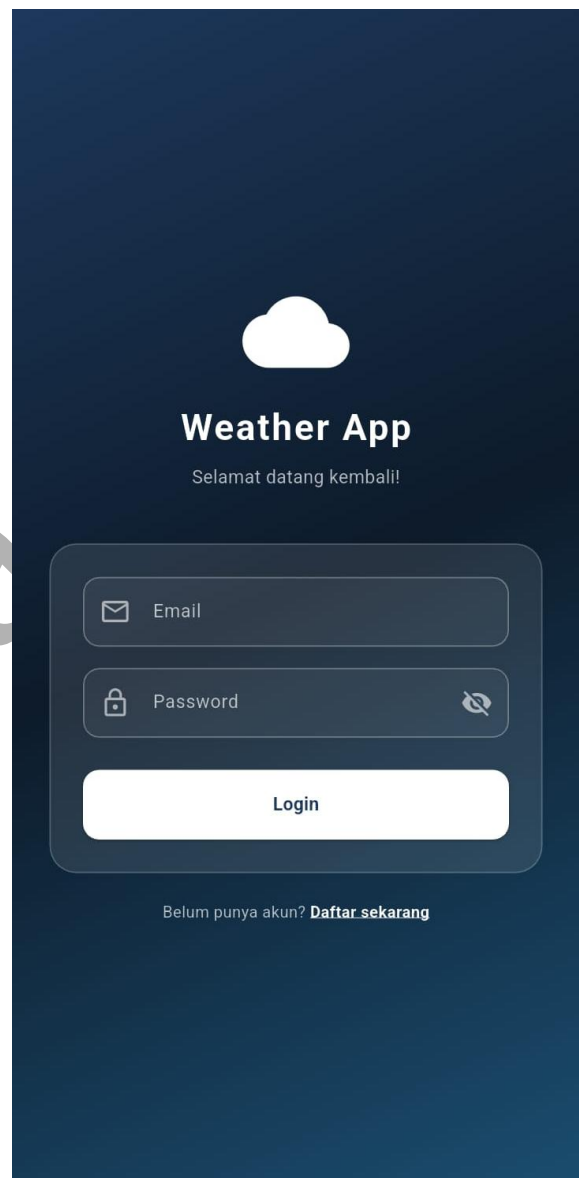
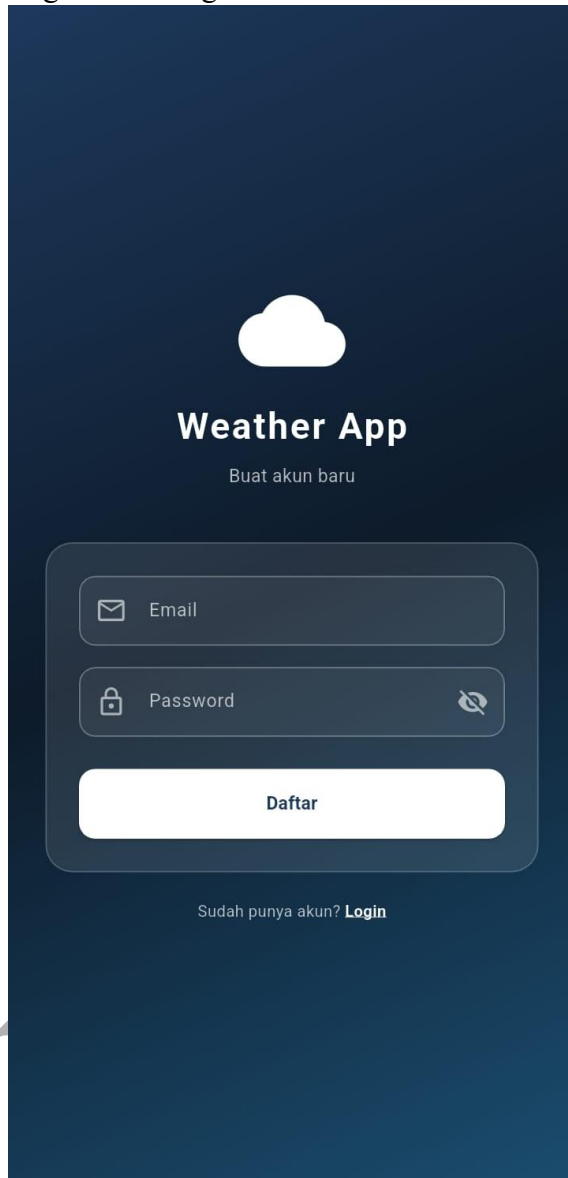


NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

Menggunakan ternary operator bersarang untuk menentukan widget mana yang ditampilkan berdasarkan state saat ini. Urutan prioritas: loading > error > data tersedia > kosong. Pola ini umum digunakan dalam Flutter untuk menangani tiga keadaan utama: loading, error, dan sukses.

Aplikasi Ketika Run di Android

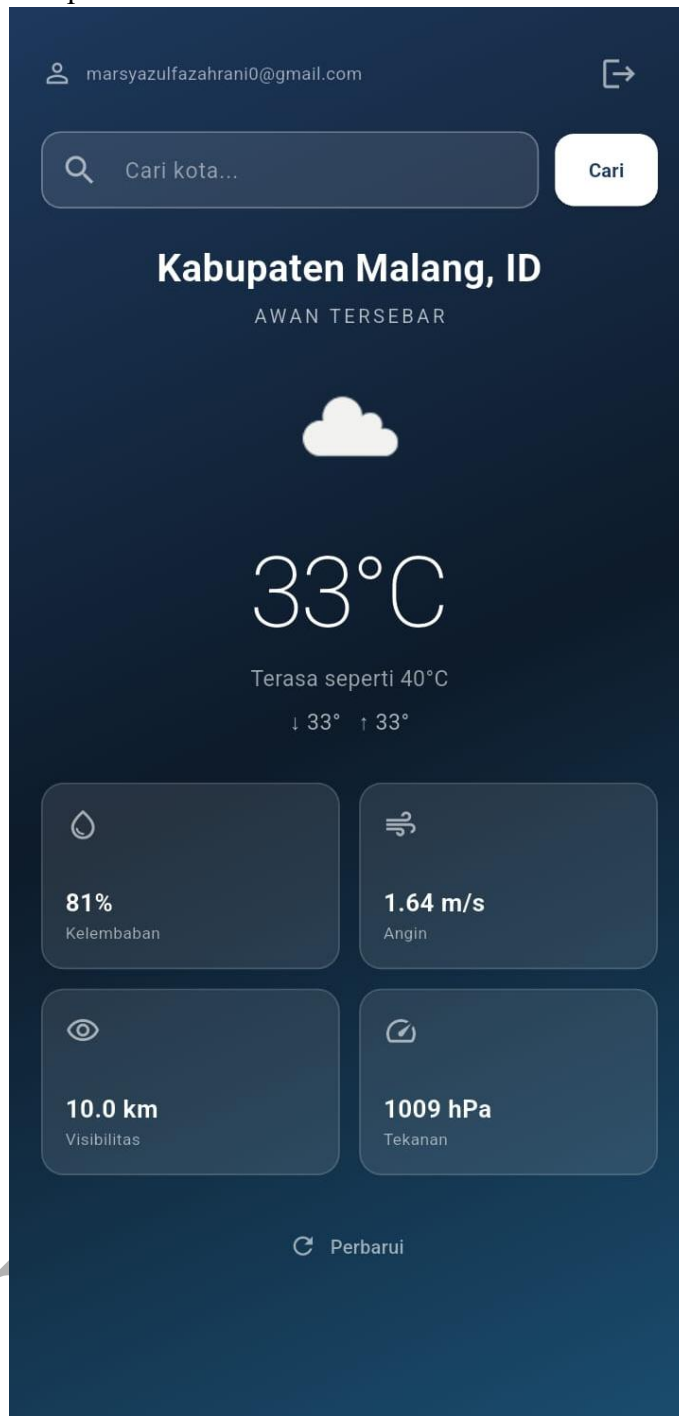
Register dan login





NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

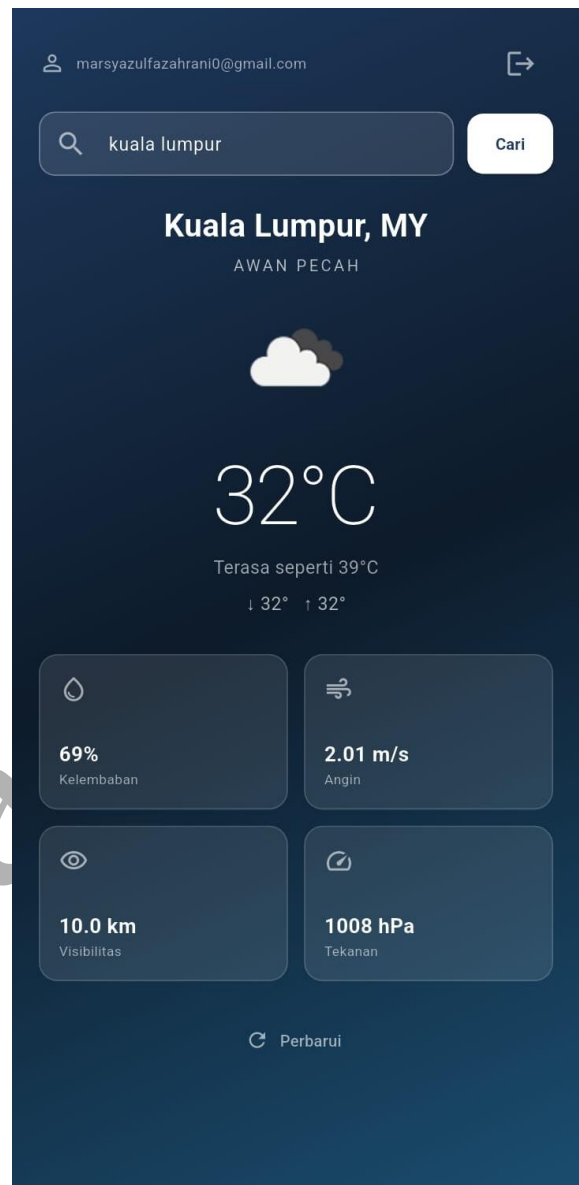
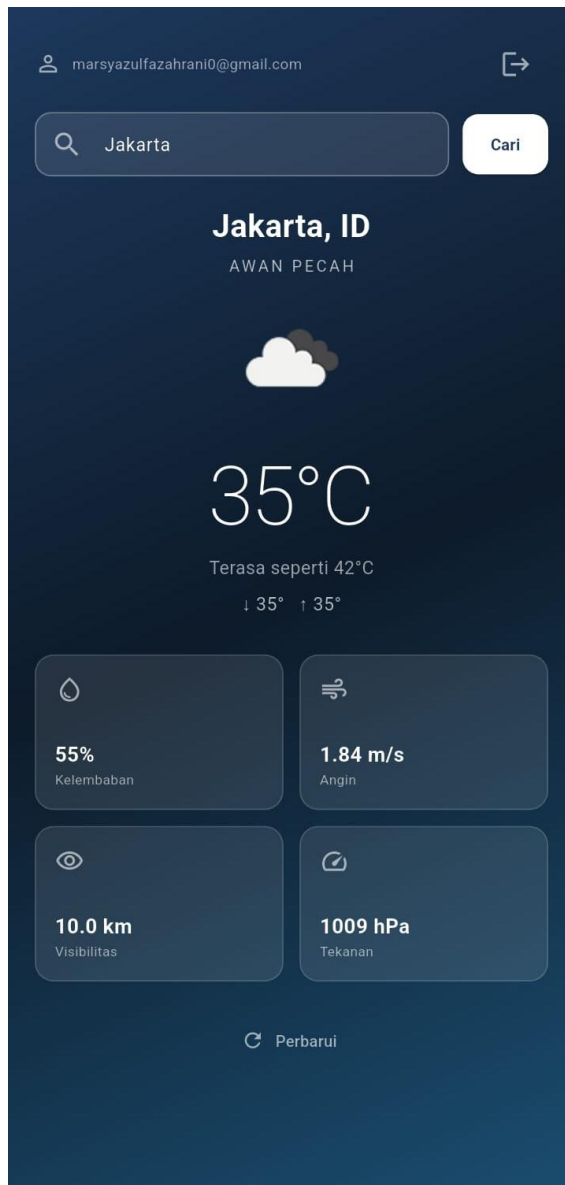
Tampilan cuaca awal





NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

Mencari cuaca kota lain





NAME : MARSYA ZULFA ZAHRANI  
NIM : 244107060005  
ABSENT : 11  
CLASS : SIB 2G  
OBJECT : MOBILE PROGRAMMING

## Logout

